



Building an AI-Powered Fault Detector for Electric Propulsion: A Hands-On Digital Engineering Workshop

Neha Sardesai

Principal Education Application Engineer
MathWorks



Innovation in Engineered Systems across Industries



Aerospace and Defense



Automotive



Communications



Software and Internet



Financial Services

Complex multi-domain systems, software-defined and autonomous, model-based and data-driven

Comms infrastructure, plus all types of connected systems across industries

Big Data, Agile, DevOps, integration with IT systems



Railway Systems



Energy Production



Electronics



Neuroscience



Biological Sciences

Modernization, often on legacy platforms, becoming data-centric for optimization and maintenance

Wide range of compute platforms, many kinds of HW/SW integration

Collaboration across roles and subject matter expertise



Process Industries



Industrial Machinery



Semiconductors



Medical Devices



Biotech and Pharmaceutical

Vertical VX4 eVTOL

Vertical Aerospace – Bristol, UK

B B C

Flying e-taxi company celebrates key milestone

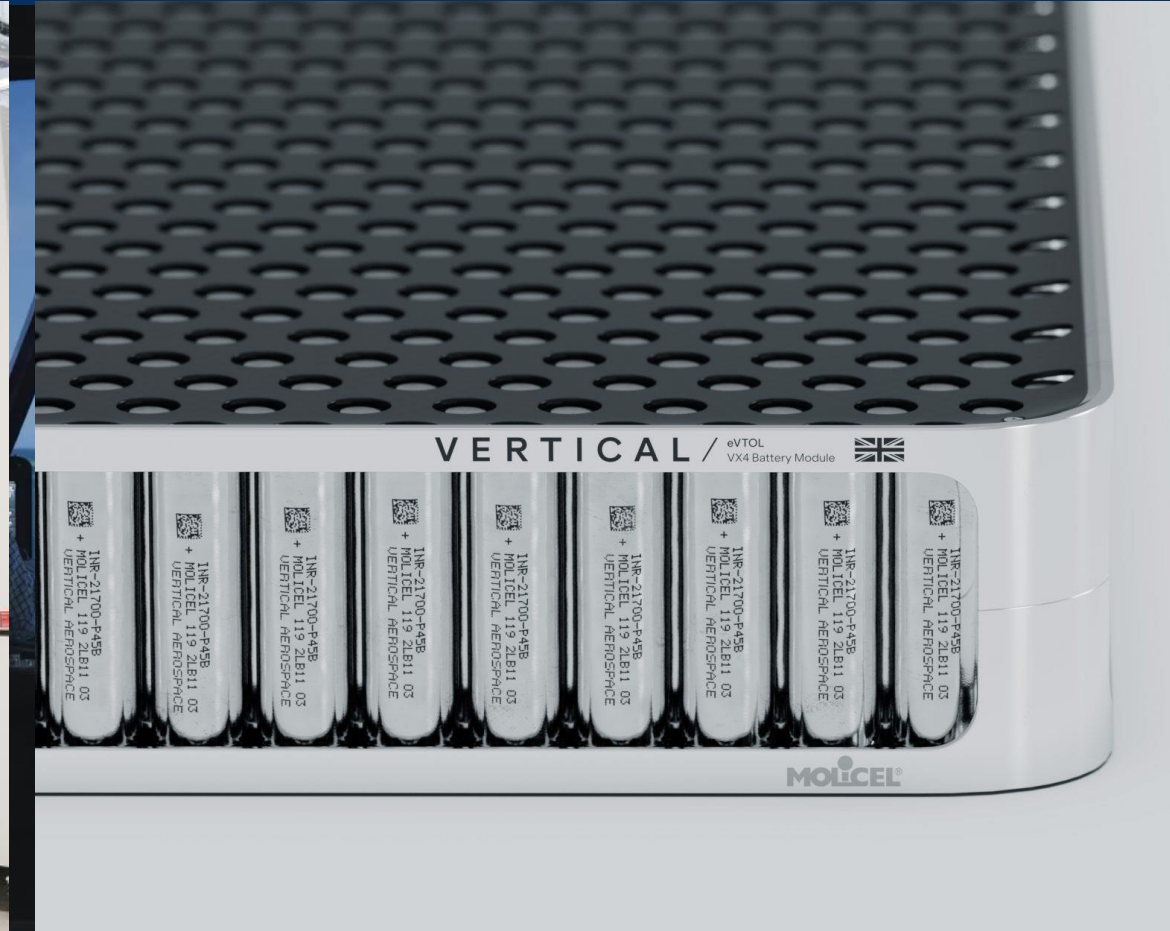
Sammy Jenkins, Bea Swallow & Theo Leggett BBC News



eVTOL =
Electric Vertical Take-off and Landing
aircraft

Vertical VX4 eVTOL

How it works



Vertical VX4 eVTOL

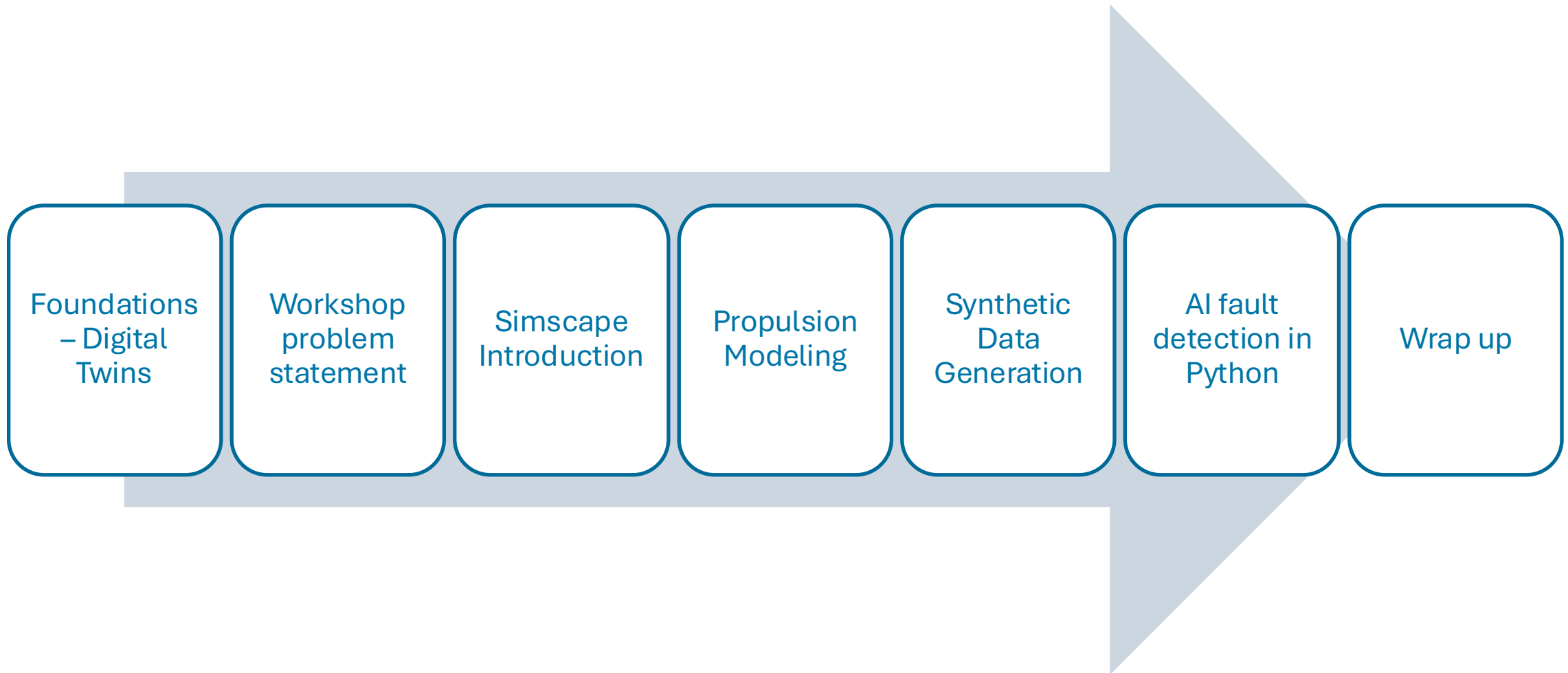
How were MATLAB and Simulink used?

- Simulink and Simscape to create aircraft model and design flight control software
- Simulink Test to verify and validate flight control software
- Simulink Real-Time and Speedgoat hardware for:
 - Hardware-in-the-loop (HIL) testing
 - Pilot-in-the-loop simulator
- System Composer for avionics design in production aircraft program
- Consulting engagements and onsite training

Chief test pilot –
“The simulator flies just like the aircraft.”

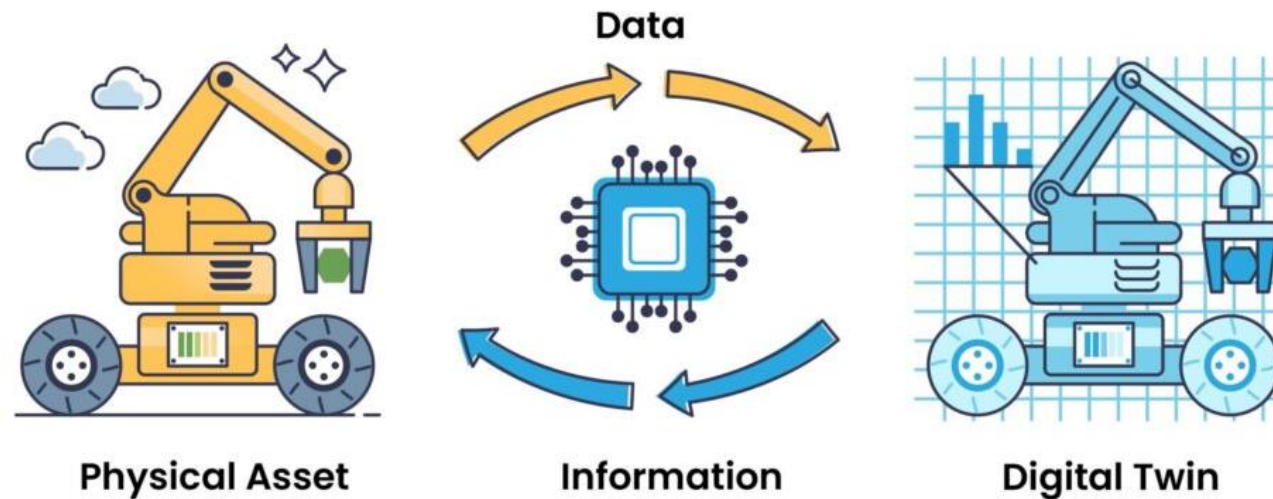


Agenda



Digital Twins

- A virtual replica of the physical system that updates in real time



- Like a flight simulator for hardware – test crashes without destroying planes

Why do Digital Twins matter?



Traditional testing: Expensive,
risky, limited scenarios

Digital twin testing: Free, safe,
unlimited scenarios



Key benefit: "Break things
virtually, not physically"

Our Use Case – Electric Propulsion



Context: Electric aircraft, UAVs, eVTOL vehicles



Challenge: Faults mid-flight can be catastrophic



Solution: Predict faults before failure



System: Battery 201.6V → Motor 20kW → Propeller 0.9m → Thrust ~139N

The Three Faults We'll Detect



Battery Fault: Internal resistance increases (aged cells, corrosion) → Voltage drops



Motor Fault: Efficiency degrades (worn bearings, winding damage) → Power loss

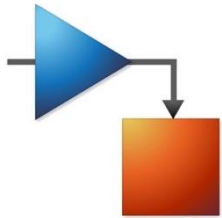


Propeller Fault: Blade damage (FOD, erosion) → Thrust reduction

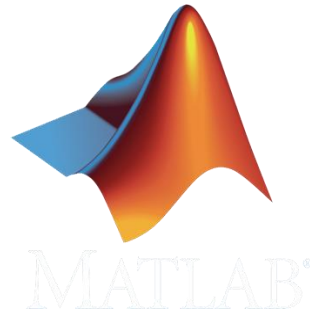


Each fault has a unique "signature" in the data

Workshop Workflow



Simscape → Build the digital twin
for electric propulsion system



MATLAB Scripts → Generate
healthy + faulty data



Python/Colab → Train ML to
classify faults

Digital Tools to use



MATLAB R2025b

MATLAB

Statistics and Machine Learning Toolbox

Simulink

Simscape

Simscape Electrical

<https://www.mathworks.com/downloads/>



Google Colab

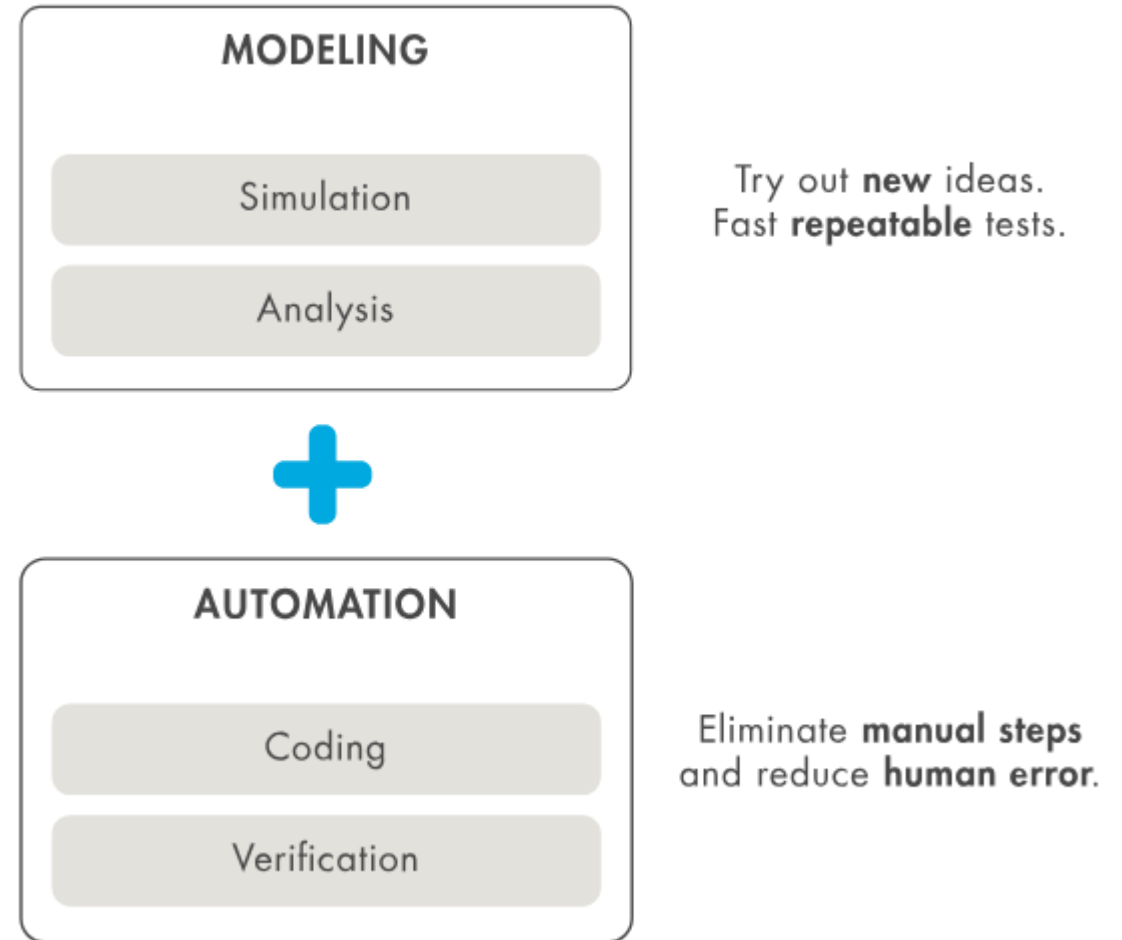
Gmail account

<https://colab.research.google.com>

What is Model-Based Design?

Creating a virtual representation of a real-world system

- **Model** new ideas and perform repeatable tests with **simulation**
- Adjust **design** based on these simulations and **iterate**

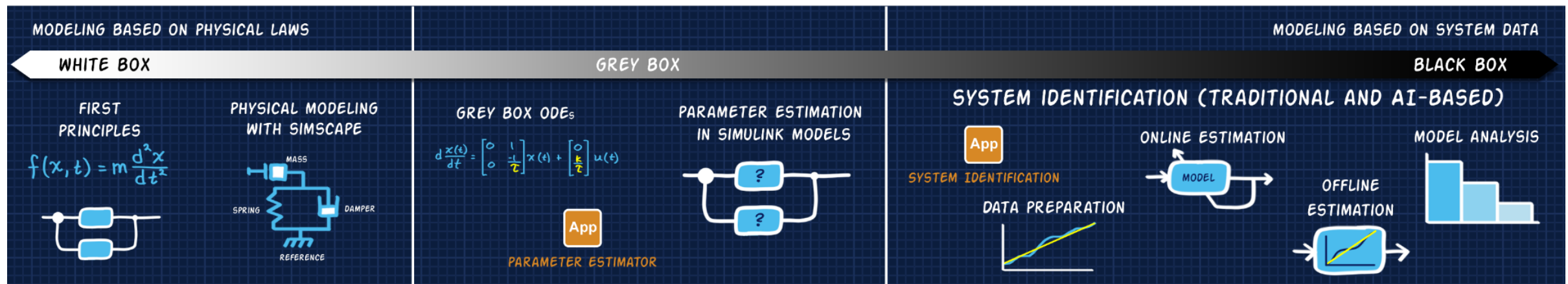


Different Approaches for Modeling Dynamic Systems

- How much/what kind of information do we know?

Model Parameters

Determine model parameters through first principles, grey box, and data-driven methods.



"I know the physical/mathematical definitions of this system, so I can determine the output behavior"

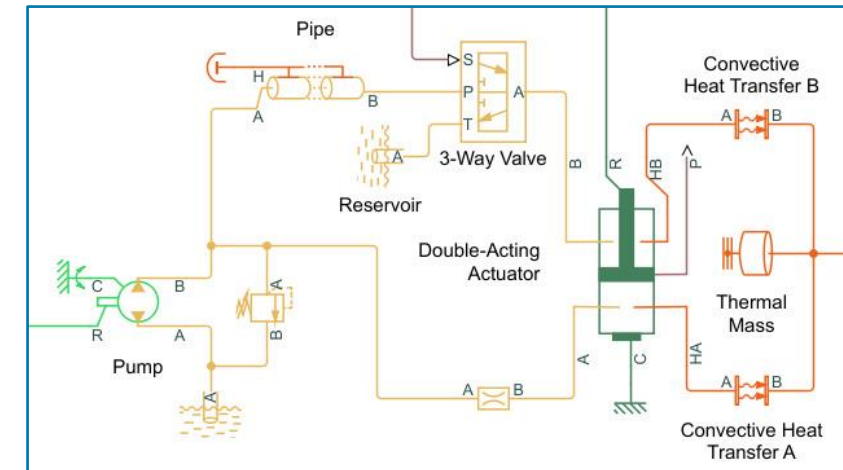
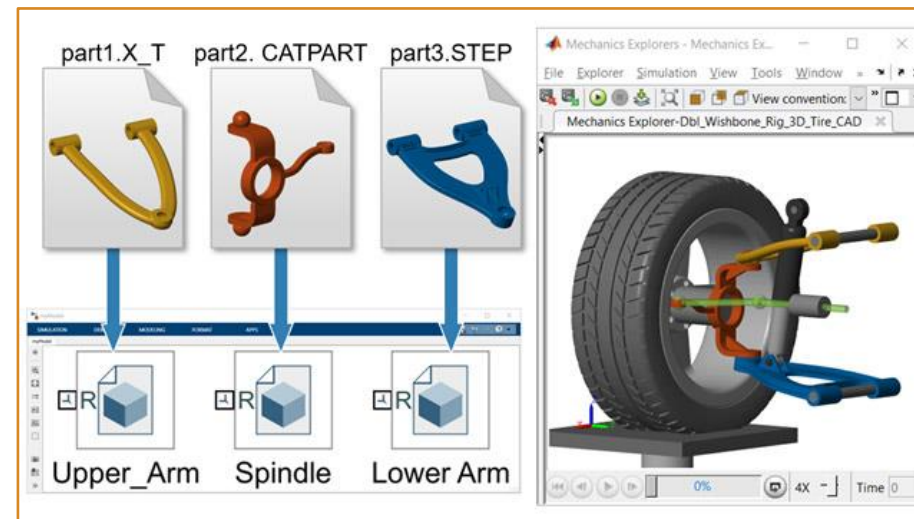
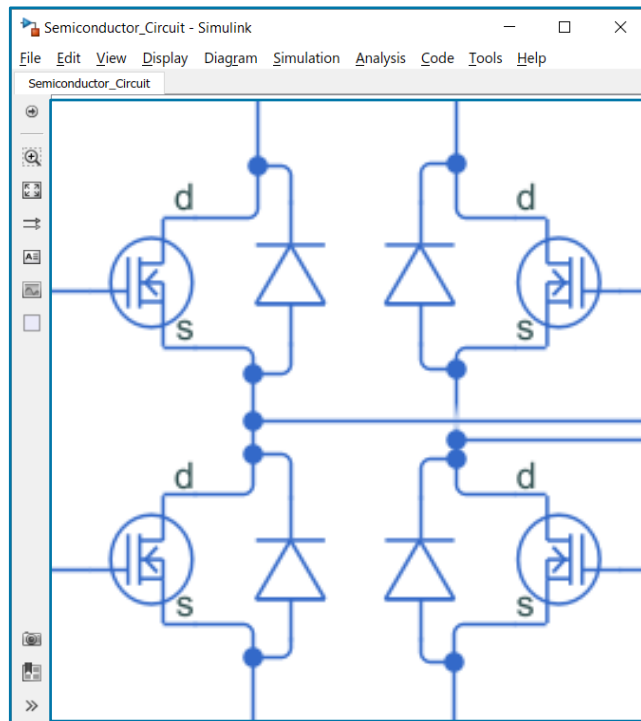
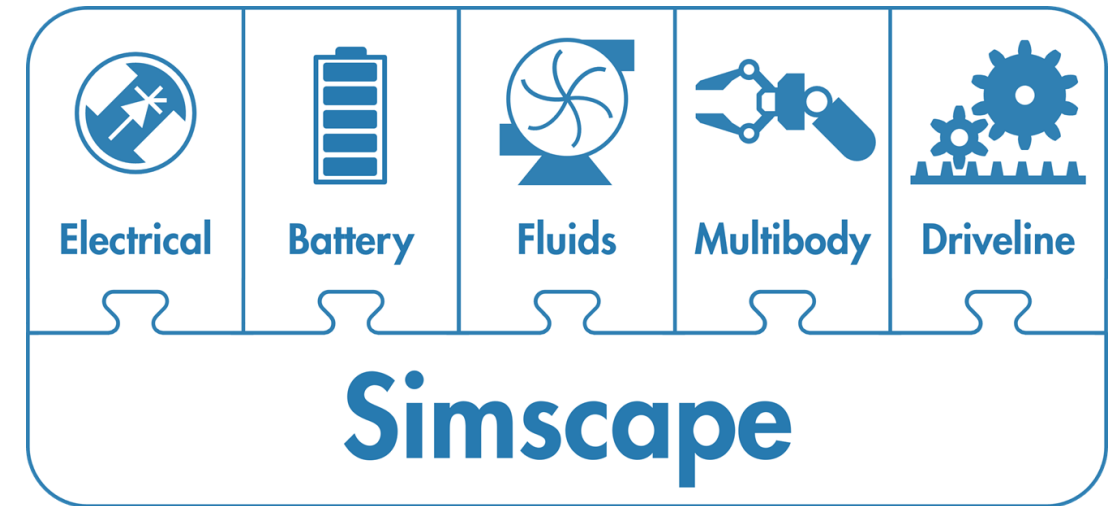


"I can measure the output behavior, so I can create a model to define the system"

Simscape

Overview

- Simscape enables you to rapidly create models of physical systems within the Simulink environment



Electric Aircraft Model in Simscape

ssc_aircraft_elec View All

Single Seat Airbus E-Fan

Electric Aircraft Component Sizing

1. [Plot battery status and currents \(see code\)](#)
2. Battery (A*hr): [100](#), [120](#), [140](#), [160](#), [180](#), [200](#)
3. Aircraft: [Single Seat](#), [Double Seat](#)
4. Flight Cycle: [Height-Based](#), [Time-Based](#)
Altitude: [Low](#), [High](#)
5. [Explore simulation results](#) using [sscexplore](#)
6. [Sweep battery sizes \(see code\)](#)
7. [Sweep payload mass \(see code\)](#)
8. [Sweep battery and payload mass \(see code\)](#)
9. [Learn more](#) about this example

ssc_aircraft_elec

▼ **Parameter Attributes**

ModelVersion	12.0
LastModifiedDate	Wed Dec 24 14:13:37 2025
LibraryLinkDisp...	disabled
ModelBrowser...	off
Dirty	off
Description	Electric Aircraft Component Sizing
Copyright 2017-2025 The MathWorks, Inc.	

Property Inspector

MATLAB, Simulink, and Simscape for Physical Modeling

Below is a simple script using MATLAB code to simulate the dynamics of a spring-mass-damper system.

```
% Parameters
m = 1; % mass (kg)
k = 10; % spring constant (N/m)
b = 1; % damping coefficient (Ns/m)

% Initial conditions
x0 = 1; % initial displacement of mass 1 (m)
v0 = 0; % initial velocity of mass 1 (m/s)

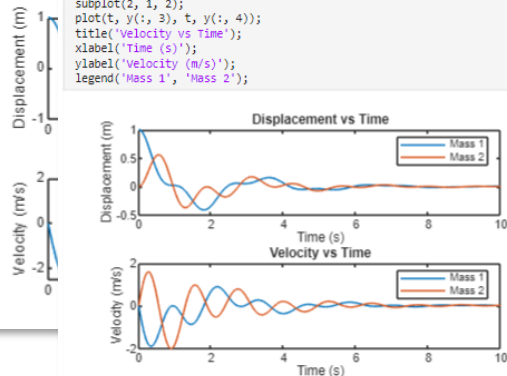
% Time span
tspan = [0 10]; % simulate from 0 to 10 seconds

% ODE function
odefun = @(t,y) [y(3); y(4);
    (-b1 * y(3) - k1 * (y(1) - y(2))) / m1;
    (-b2 * y(4) + k1 * (y(1) - y(2)) - k2 * y(2)) / m2];

% Solve ODE
[t, y] = ode45(odefun, tspan, [x1_0; x2_0; v1_0; v2_0]);

% Plot results
figure;
subplot(2,1,1);
plot(t, y(:,1), t, y(:,2));
title('Displacement vs Time');
xlabel('Time (s)');
ylabel('Displacement (m)');
legend('Mass 1', 'Mass 2');

subplot(2,1,2);
plot(t, y(:,3), t, y(:,4));
title('Velocity vs Time');
xlabel('Time (s)');
ylabel('Velocity (m/s)');
legend('Mass 1', 'Mass 2');
```



MATLAB

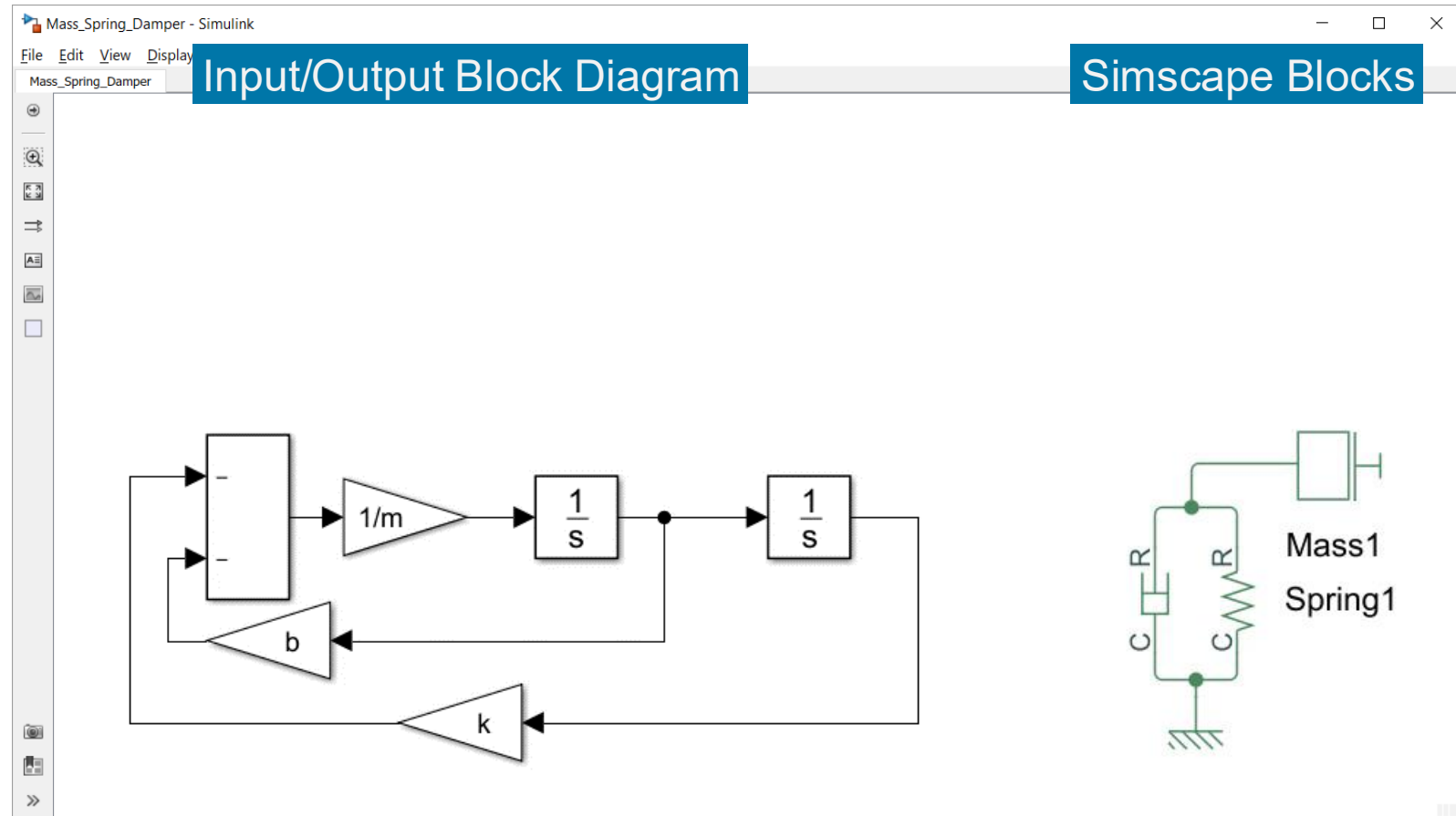
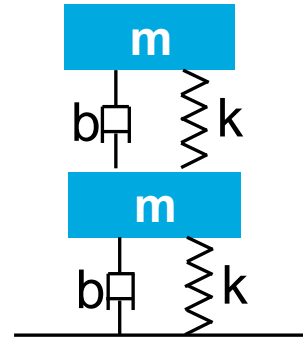
$$F_{\text{Spring}} = k_{\text{Spring}} * (x_{\text{Mass}})$$

$$F_{\text{Damper}} = b_{\text{Damper}} * \left(\frac{dx_{\text{Mass}}}{dt}\right)$$

$$\frac{d^2 x_{\text{Mass}}}{dt^2} = \frac{-F_{\text{Spring}} - F_{\text{Damper}}}{m_{\text{Mass}}}$$

$$F_{\text{Spring2}} = k_{\text{Spring}} * (x_{\text{Mass}})$$

....



Simulink

Simscape Battery Onramp

 MATLAB Help Center Community **Learning** [Get MATLAB](#) [Sign In](#)  

Online Courses

[Home](#) | [My Courses](#) | [Online Training Suite](#) | [Documentation & Support](#) ▼



Simscape Battery Onramp

[Take course](#) [Share Course](#)

Learn the basics of simulating a simple battery management system (BMS) for safe charging/discharging in various temperatures. Use Simscape to simulate battery packs and their heat exchange and algorithms like coulomb counting and constant-current (CC) constant-voltage (CV) charging.

Course modules

- [Course Overview](#) 10 min
- ▼ Battery Discharging Circuit** 5 min
 - Familiarize yourself with Simscape Battery.
 - Lessons:
 - Battery Discharging Circuit
- [Battery Pack Modeling](#) 25 min

About this course



Format: [Self-paced](#)
Length: About 1 hour
Language: English

Recommended prerequisites

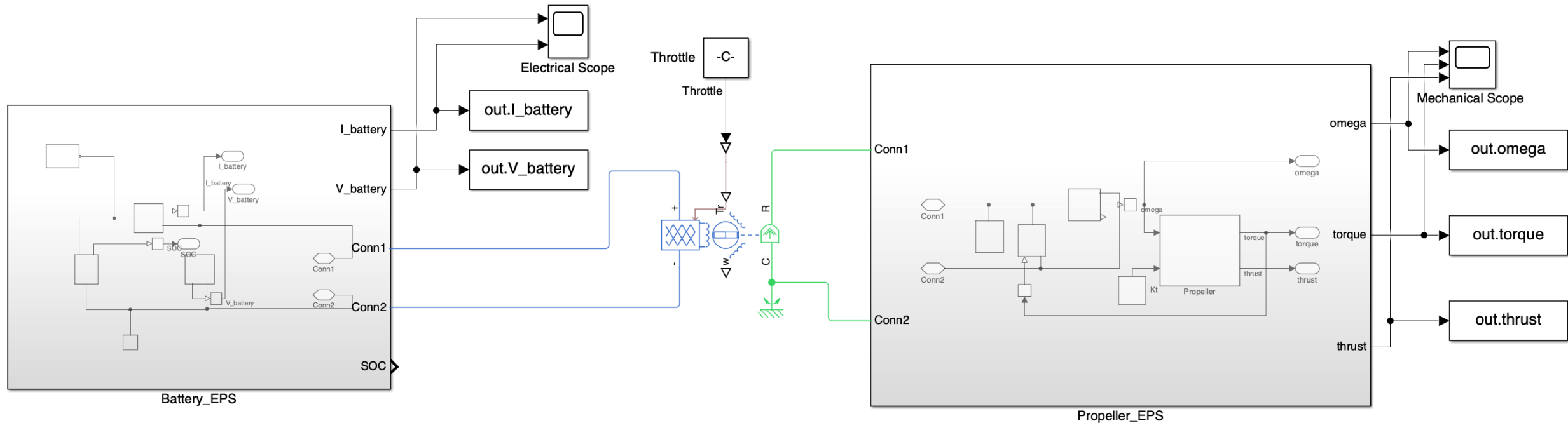
-  [Simulink Onramp](#)
-  [Simscape Onramp](#)

Features

- Hands-on exercises with automated feedback

Part 1: Building the Simscape Model

Estimated time: 90 minutes — Goal: Create propulsion model



GitHub Repository for workshop resources

<https://github.com/nehasardesai/Electric-Propulsions-System-Digital-Twin>

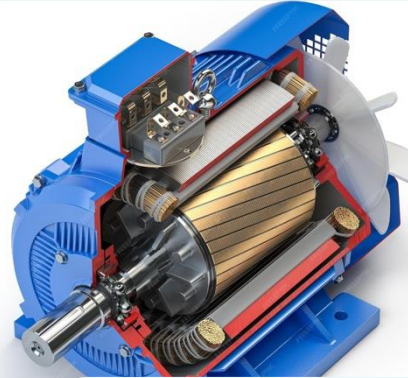


 nehasardesai Workshop instructions.		
 resources/project		Commit.
 .gitattributes		First commit.
 .gitignore		First commit.
 EPS_Fault_Detection.ipynb		Dataset gen and ML.
 ElectricPropSys_digitalTwin.slx		Deleted files.
 ElectricPropulsionSystem_DigitalTwin.prj		First commit.
 FaultDatasetGen_Simscape.mlx		Updates.
 ModelSetupInstructions.m		Dataset gen and ML.
 README.md		Update workshop agenda duration for Part 1
 Workshop_Instruction_Manual.docx		Workshop instructions.
 Workshop_Preview_Instructions.docx		Datasets and instructions.

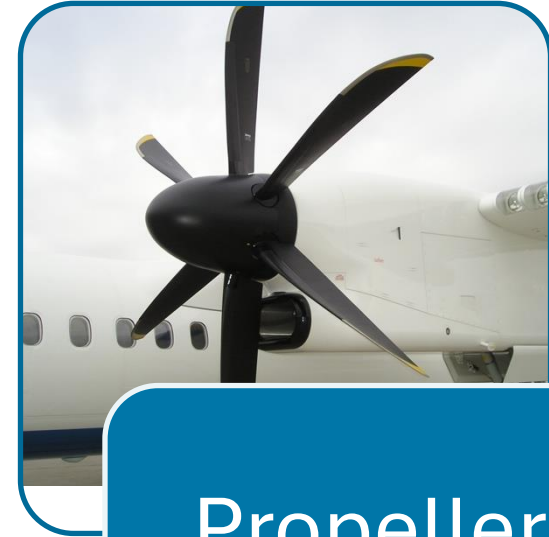
Block Diagram and System Specifications



Battery
201.6V



Motor
20kW, 88%
efficiency



Propeller
0.9m
diameter

Realistic for small electric aircraft

Live Demo - Creating the Model

The Propeller Function



Thrust equation: $T = K_t \times \rho \times n^2 \times D^4$

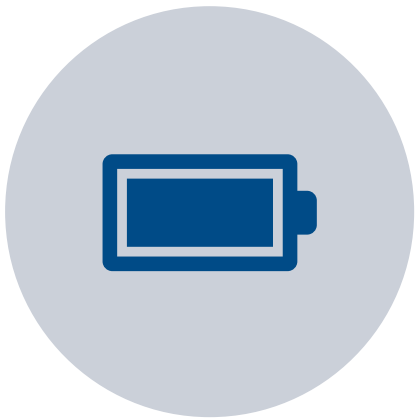


Analogy: Fan—faster spin & bigger blades = more thrust



Fault injection through `Kt_input` (later)

Wiring the Model



ELECTRICAL: BATTERY →
CURRENT SENSOR → MOTOR



MECHANICAL: MOTOR →
PROPELLER LOAD



SIGNALS LOGGED USING TO
WORKSPACE BLOCKS

Part 2: Inject Faults and Generate Training Data

Estimated time: 30 minutes

Fault Label	Throttle	Mean Voltage	Mean Current	Mean Speed	Mean Torque	Mean Thrust	Electrical Power	Mechanical Power	Efficiency
0	0.9630332	195.60343	114.80386	2469.3663	-74.465981	137.89997	22456.028	19256.863	85.753644
0	0.9054104	195.61033	114.67692	2468.1424	-74.413871	137.80347	22431.991	19233.85	85.742947
0	0.3960244	199.48517	40.505087	1771.6157	-38.418735	71.145805	8080.1641	7127.7867	88.21339
0	0.5112763	198.4407	60.505316	2014.4867	-49.679948	91.999904	12006.717	10480.636	87.289774
0	0.4473473	199.04141	49.00388	1882.5712	-43.405347	80.380272	9753.8014	8557.2999	87.732972
0	0.8186342	195.62741	114.36194	2464.5253	-74.25773	137.51431	22372.329	19165.363	85.665481
0	0.4469997	199.04452	48.944337	1881.843	-43.371714	80.317989	9742.1022	8547.3616	87.736316
0	0.3712866	199.68892	36.60356	1714.2379	-35.985642	66.640077	7309.3255	6460.1479	88.382271
0	0.7864171	195.63595	114.20419	2463.0802	-74.189938	137.38877	22342.445	19136.639	85.651501
0	0.8242285	195.62619	114.3844	2464.7294	-74.267011	137.5315	22376.584	19169.346	85.666988
0	0.5203066	198.35249	62.195194	2032.0697	-50.553294	93.617211	12336.572	10757.966	87.203854
0	0.7147909	196.2108	103.20272	2386.2227	-69.645184	128.97256	20249.489	17403.803	85.946873
0	0.5797722	197.7513	73.714108	2142.3162	-56.233747	104.13657	14577.061	12616.03	86.547143
0	0.7051146	196.32759	100.96679	2369.8574	-68.695068	127.21309	19822.566	17048.646	86.006249
0	0.8639215	195.61647	114.56286	2467.1277	-74.370531	137.72321	22410.383	19214.745	85.740366
0	0.9231057	195.60807	114.71857	2468.5679	-74.432485	137.83794	22439.878	19241.977	85.74903
0	0.5235377	198.32057	62.806436	2038.4018	-50.868444	94.200822	12455.808	10858.763	87.178309
0	0.7685243	195.64139	114.10371	2462.4884	-74.154258	137.3227	22323.408	19122.84	85.662729
0	0.9275938	195.60748	114.72925	2468.6653	-74.436533	137.84543	22441.9	19243.783	85.749349
0	0.4550129	198.97794	50.231688	1894.6909	-44.014907	81.509087	9994.9976	8733.3378	87.377087
0	0.6877185	196.53468	97.001158	2340.2296	-66.991168	124.05772	19064.092	16417.919	86.119598
0	0.3705788	199.69463	36.494152	1712.57	-35.916062	66.511226	7287.6863	6441.3835	88.387223
0	0.5236797	198.31918	62.833263	2038.6765	-50.882145	94.226195	12461.041	10863.151	87.176918
0	0.4617685	198.91673	51.403642	1908.56	-44.664138	82.711367	10225.044	8927.0276	87.305516
0	0.8938312	195.61193	114.64698	2467.8856	-74.40314	137.78359	22426.317	19229.075	85.743349

GitHub Repository for workshop resources

<https://github.com/nehasardesai/Electric-Propulsions-System-Digital-Twin>



 nehasardesai Workshop instructions.		
 resources/project		Commit.
 .gitattributes		First commit.
 .gitignore		First commit.
 EPS_Fault_Detection.ipynb		Dataset gen and ML.
 ElectricPropSys_digitalTwin.slx		Deleted files.
 ElectricPropulsionSystem_DigitalTwin.prj		First commit.
 FaultDatasetGen_Simscape.mlx		Updates.
 ModelSetupInstructions.m		Dataset gen and ML.
 README.md		Update workshop agenda duration for Part 1
 Workshop_Instruction_Manual.docx		Workshop instructions.
 Workshop_Preview_Instructions.docx		Datasets and instructions.

How We Inject Faults

Programmatically changing parameters

Battery

- $R = 0.05 \rightarrow 0.15 \Omega$

Motor: Efficiency

- $88\% \rightarrow 70\%$

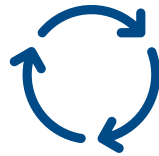
Propeller

- $K_t = 0.10 \rightarrow 0.06$

Data Generation Script



1. Loads model



2. Loops through
faults



3. 50 simulations
per fault



4. Extracts features
and saves CSV

Dataset Structure



Features: Throttle, Voltage, Current, Speed, Torque, Thrust, Power, Efficiency

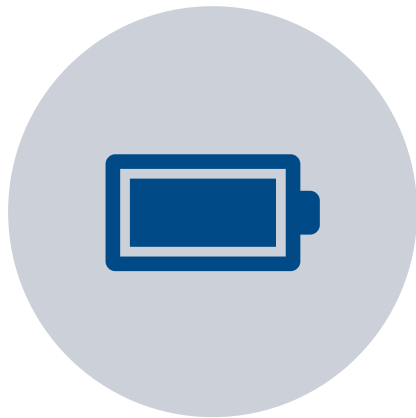


Labels: 0=Healthy, 1=Battery, 2=Motor, 3=Propeller



~200 samples

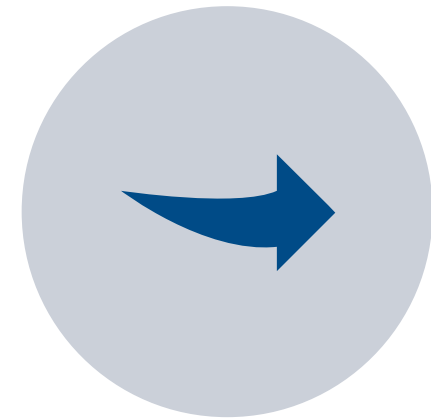
Visualizing Fault Signatures



BATTERY FAULT → LOWER
VOLTAGE



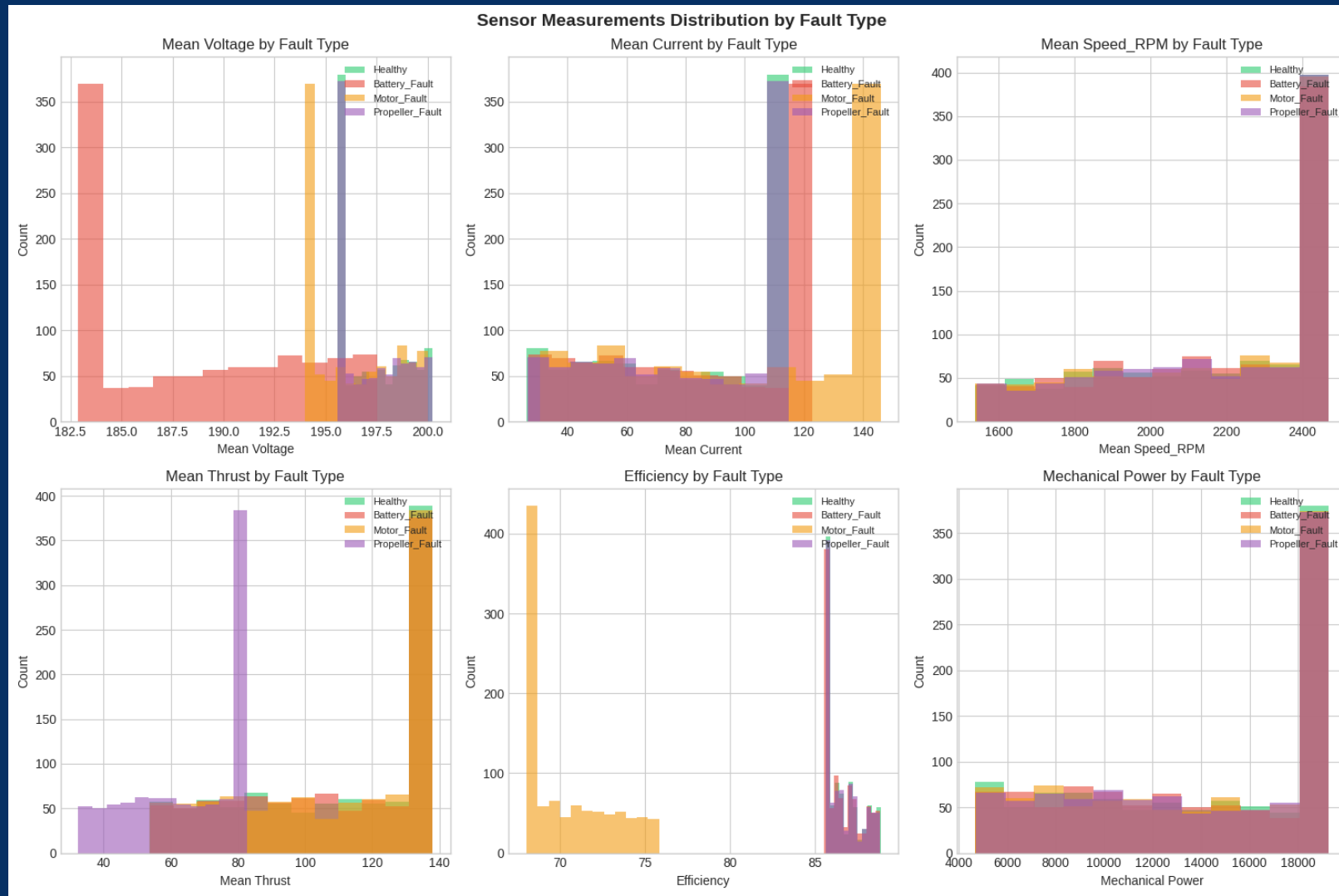
MOTOR FAULT → LOWER
EFFICIENCY, HIGHER
CURRENT



PROPELLER FAULT →
LOWER THRUST FOR SAME
RPM

Part 3: Machine Learning Model for Fault Prediction

Estimated time: 30 minutes



GitHub Repository for workshop resources

<https://github.com/nehasardesai/Electric-Propulsions-System-Digital-Twin>



 nehasardesai Workshop instructions.		
 resources/project		Commit.
 .gitattributes		First commit.
 .gitignore		First commit.
 EPS_Fault_Detection.ipynb		Dataset gen and ML.
 ElectricPropSys_digitalTwin.slx		Deleted files.
 ElectricPropulsionSystem_DigitalTwin.prj		First commit.
 FaultDatasetGen_Simscape.mlx		Updates.
 ModelSetupInstructions.m		Dataset gen and ML.
 README.md		Update workshop agenda duration for Part 1
 Workshop_Instruction_Manual.docx		Workshop instructions.
 Workshop_Preview_Instructions.docx		Datasets and instructions.

Machine Learning in Python on Colab



Go to <https://colab.research.google.com>

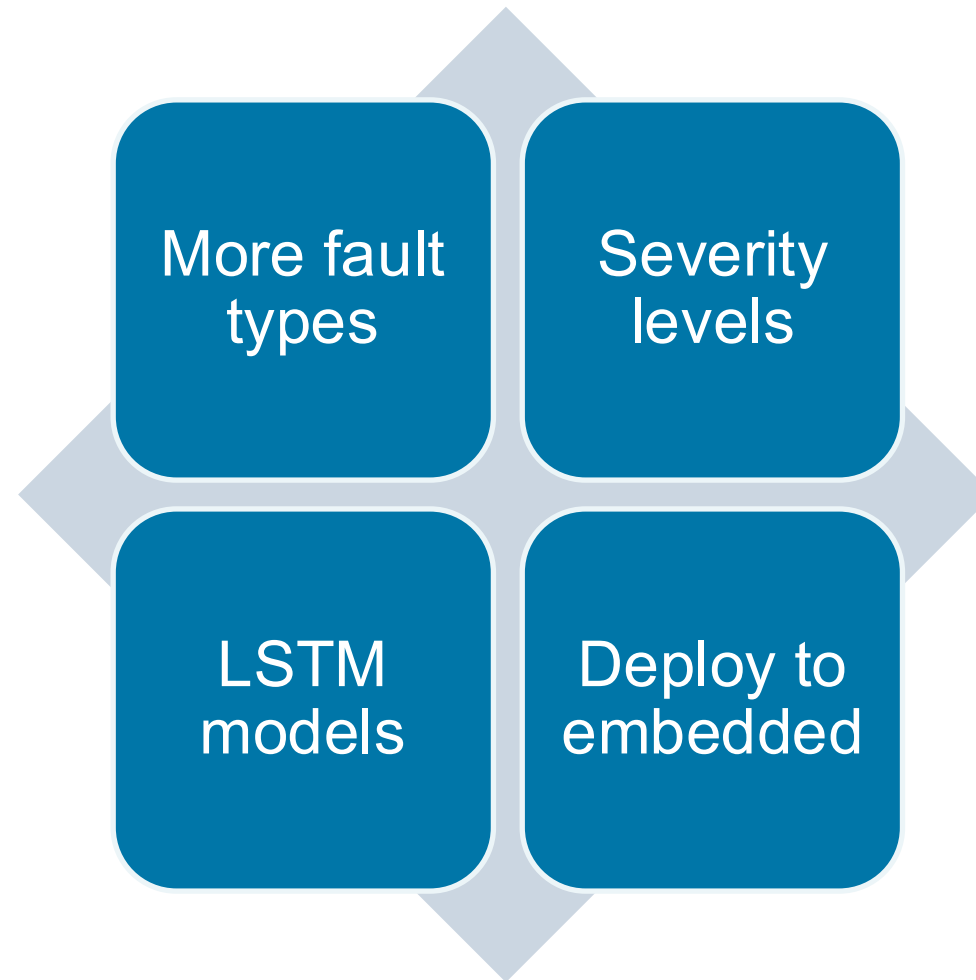


Login with your google ID

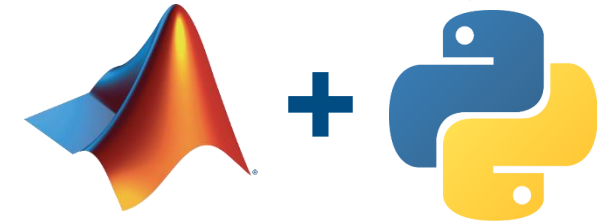


Upload notebook (link in GitHub repo) and dataset (use your dataset first)

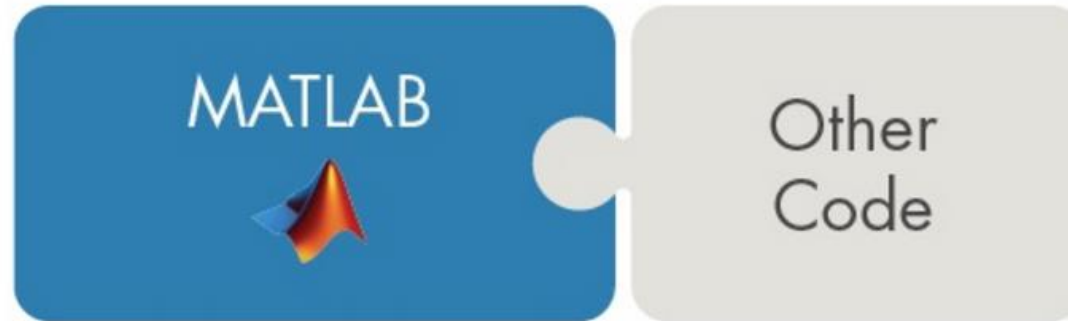
Next Steps



What we would do in a real situation



Call Libraries Written in Another Language

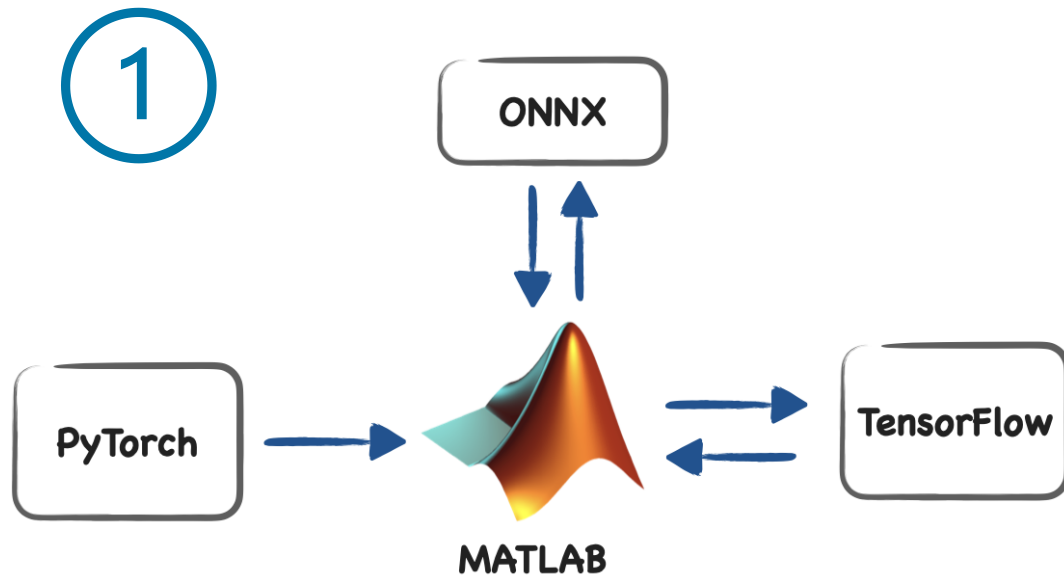


Call MATLAB from Another Language

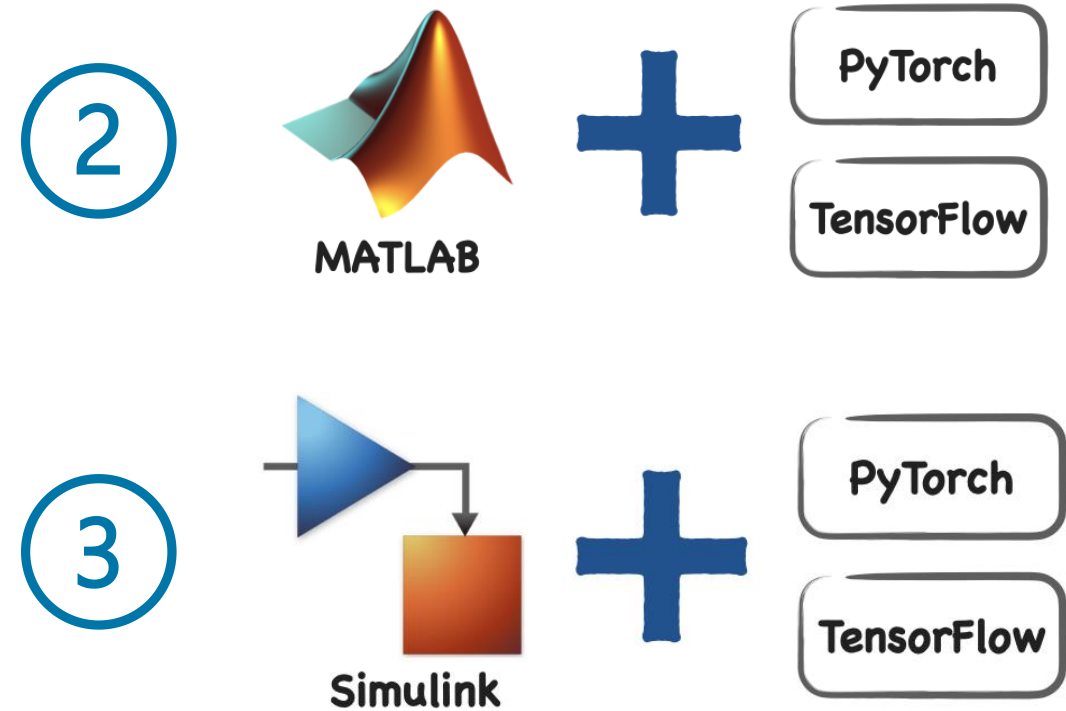


3 Ways to Integrate with PyTorch and TensorFlow

Model Conversion between MATLAB, PyTorch, and TensorFlow



Co-execution of PyTorch and TensorFlow models in **MATLAB** and **Simulink**



Get Started for Free with Onramp Courses



MATLAB Onramp

Get started quickly with the basics of MATLAB®.

[Details and launch](#)



Simulink Onramp

Get started quickly with the basics of Simulink®.

[Details and launch](#)



Image Processing Onramp

Learn the basics of practical image processing techniques in MATLAB.

[Details and launch](#)



Signal Processing Onramp

An interactive introduction to practical signal processing methods for spectral analysis.

[Details and launch](#)



Machine Learning Onramp

An interactive introduction to practical machine learning methods for classification problems.

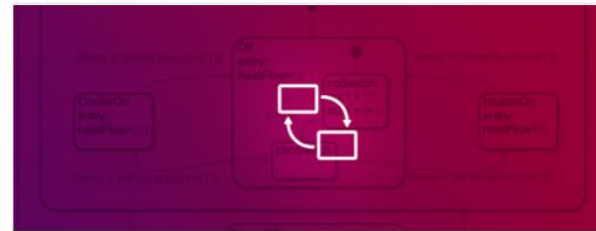
[Details and launch](#)



Deep Learning Onramp

Get started quickly using deep learning methods to perform image recognition.

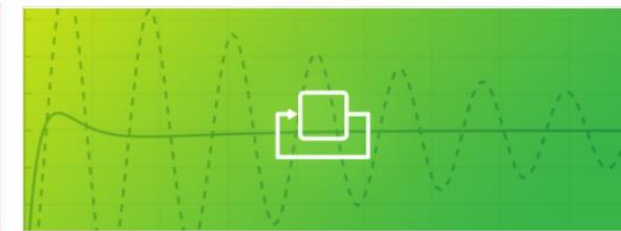
[Details and launch](#)



Stateflow Onramp

Learn the basics of creating, editing, and simulating state machines in Stateflow®.

[Details and launch](#)



Control Design Onramp with Simulink

Get started quickly with the basics of feedback control design in Simulink.

[Details and launch](#)

